# Social4CSE: A Scalable Publish-Subscribe System with Advanced Features

Bruno Picoto (113197) Lídia Coelho (117345)

***Abstract*—This paper presents the design, implementation, and evaluation of Social4CSE, a scalable publish-subscribe communication system that enables real-time content dissemination across multiple users. The system employs a client-server architecture with a centralized multi-threaded server managing user authentication, topic subscriptions, and message distribution. Key innovations include hybrid cryptographic protocols combining RSA and AES encryption for secure communication, efficient concurrency management using fine-grained locking mechanisms, and a web-based administrative interface for system monitoring. Performance evaluation demonstrates that the system can handle multiple concurrent users with acceptable latency while maintaining data consistency and security. The implementation addresses critical distributed systems challenges including concurrent access control, secure communication, and fault tolerance through persistent logging.**

## I. Introduction

Modern distributed systems require efficient mechanisms for disseminating information among large numbers of participants while maintaining decoupling between producers and consumers. The publish-subscribe (pub/sub) communication model addresses this need by allowing users to publish content to topics and subscribe to receive updates from those topics [**?**]. This decoupling enhances scalability and flexibility, making pub/sub systems ideal for applications ranging from social networks to financial trading platforms.

The Social4CSE project implements a comprehensive pub/sub system that combines theoretical principles with practical implementation challenges. The system architecture employs a centralized server that maintains global state including user accounts, topic hierarchies, and message history. Clients connect to this server via TCP connections, exchanging JSON-formatted messages over encrypted channels. This design choice simplifies consistency management while providing a foundation for exploring advanced distributed systems concepts.

This paper makes three primary contributions: (1) a detailed implementation of a secure pub/sub system with hybrid cryptographic protocols, (2) an evaluation of concurrency control strategies for multi-threaded server environments, and (3) a comprehensive analysis of system performance under varying load conditions. The remainder of this paper is organized as follows: Section II presents the system architecture, Section III details server implementation, Section IV discusses client design, Section V covers advanced features including security and persistence, Section VI evaluates system performance, and Section VII concludes with directions for future work.

## II. System Architecture

Social4CSE follows a centralized client-server architecture where all system state is maintained by a single server process. This design choice simplifies consistency management and provides a single source of truth for all operations.

### A. Server Architecture

The server is implemented as a multi-threaded TCP server that listens for incoming client connections on port 9999. Each client connection is handled by a dedicated thread, allowing concurrent processing of multiple client requests. The server maintains three primary data structures in memory:

- `user_database`: Dictionary mapping usernames to user objects containing hashed passwords, subscription lists, and metadata

```
{...,"username":{"password":password,"topics":{"
    topic1":last viewed post ID,"topic2":last
    viewed post ID}},...}
```

- `topics`: Nested dictionary structure organizing posts by topic with unique post identifiers

```
{...,"topic1": {...,post_id:{"post":"abc","
    author":"name1","metadata":{"id":post_id, '
    reply_to': a post_id or None}},...},...}
```

- `active_tokens`: Session management mapping authentication tokens to active user sessions

To ensure thread safety, the system employs fine-grained locking with separate locks for user data (`user_lock`), topic data (`topic_lock`), and session tokens (`token_lock`). This approach minimizes contention compared to a single global lock while maintaining data consistency.

### B. Client Architecture

Clients provide user interfaces through two complementary mechanisms: a command-line interface for direct interaction and a web-based interface built with Flask. The dual-interface approach caters to different user preferences and use cases. The command-line client supports both single-request operations and interactive sessions, while the web interface offers a more intuitive user experience with real-time updates.

### C. Communication Protocol

All client-server communication follows a structured protocol combining length-prefixed message framing with JSON serialization. Each message consists of a 4-byte length header followed by the JSON payload. This design ensures reliable

message boundaries while maintaining human-readable message content. The protocol supports 9 distinct operations as shown in Table I.

TABLE I: Supported Operations in Social4CSE Protocol

| Operation | Parameters | Description |
|---|---|---|
| register | username, password | Create new user account |
| login | username, password | Authenticate and receive session token |
| subscribe | token, topic | Subscribe to specified topic |
| publish | token, topic, content | Publish message to topic |
| refresh | token, topic, flag | Retrieve new/unseen messages |
| listusers | - | List all registered users |
| listtopics | - | List all topics with statistics |
| stats | - | Retrieve system performance metrics |
| profile | username | Get user profile information |

## III. Server Implementation Details

### A. Concurrency Management

The server's multi-threaded design necessitates careful synchronization to prevent race conditions and ensure data consistency. Rather than using a single global lock, the implementation employs multiple fine-grained locks that protect specific data structures while avoiding deadlocks, re-acquisitions, and nested locks:

```
user_lock = threading.Lock()
topic_lock = threading.Lock()
token_lock = threading.Lock()
stats_lock = threading.Lock()

def publish(ut, t, p, reply):
    with token_lock:
        if not valid_tokenQ(ut):
            return "Failure"
        user = user_from_token(ut)

    with user_lock:
        if t not in user_database[user]["topics"]:
            return "Failure"

    with topic_lock:
        if t not in topics:
            return "Failure"
        # ... publish operation
```

Listing 1: Example Fine-Grained Locking Implementation

This approach allows concurrent operations on different data structures while serializing accesses to the same structure. For example, user registration can proceed concurrently with topic listing, but multiple publish operations to the same topic are serialized.

### B. Authentication and Session Management

User authentication employs a token-based mechanism where successful login generates a cryptographically secure 16-byte token using Python's `secrets` module. Tokens are stored in the `active_tokens` dictionary and validated for subsequent requests. This approach avoids transmitting passwords repeatedly and enables session revocation by simply removing the token from the active tokens dictionary.

### C. Message Persistence

To provide fault tolerance and enable system recovery, all critical operations are logged to persistent storage. User-related events (registration, subscription) are logged to `users.log`, while topic operations (publish) are logged to topic-specific files in the `topics/` directory. The server supports two startup modes:

- **Fresh start**: Initializes empty data structures and clears existing logs
- **Reboot**: Recovers system state from persistent logs, allowing graceful recovery from crashes

## IV. Client Design and Implementation

### A. Dual-Interface Approach

Social4CSE provides two client interfaces catering to different user preferences:

**Command-line Client:** Implemented in `client_lib.py`, this client offers direct TCP communication with the server. It supports both single-command execution and interactive sessions where users can publish messages and receive real-time updates.

**Web Interface:** Built using Flask, the web interface provides a modern user experience with real-time updates, topic management, and system monitoring capabilities. The interface runs on a configurable port and communicates with the core server via the same protocol as the command-line client.

It comes with new features such as an easy topic switcher and subscription system, older message viewing on login and topic switches (shows past 200 messages, even if they have already been seen) providing the user with more context to the ongoing conversations. Additionally we implemented message replying, initiated by clicking any previous message in the feed:

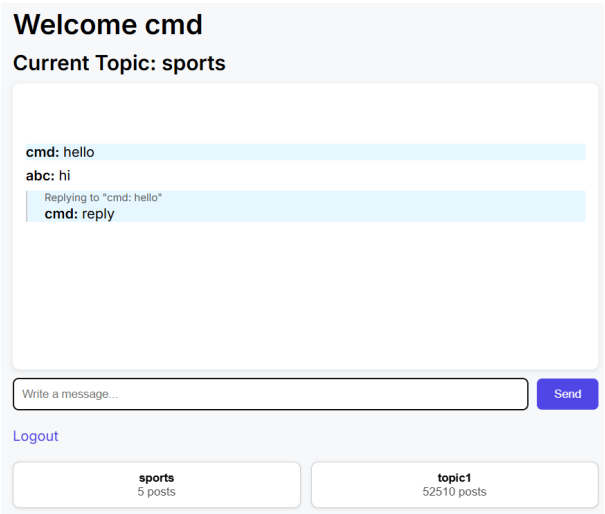



Fig. 1: Message replying example, the "Replying to: ..." text is clickable, and scrolls the feed to the target message, highlighting it.

### B. Secure Communication Protocol

Client-server communication employs a hybrid cryptographic approach combining RSA for key exchange and AES for bulk data encryption:

1) **Key Exchange:** Server generates RSA keys, sends public key to client, client generates random AES key, sends it to the server encrypted with server's public key
2) **Symmetric Encryption:** All subsequent messages are encrypted using AES-GCM mode providing both confidentiality and integrity
3) **Message Framing:** Encrypted messages are prefixed with 4-byte length headers for reliable transmission

### C. Concurrent Operations in Interactive Mode

The command-line client uses two concurrent threads in interactive mode to maintain responsiveness:

- `stdin_reader`: Monitors user input for publish commands
- `refresher`: Periodically polls server for new messages every 100ms

A shared socket lock ensures thread-safe access to the network connection, preventing interleaved reads and writes.

## V. Advanced Features

### A. Web-based Administration Interface

The system includes a comprehensive web administration interface built with Flask that provides:

- Real-time system statistics including user count, topic count, and operation latency
- Visual monitoring through dynamically generated plots
- User activity monitoring and session management

### B. Performance Monitoring and Statistics

The server continuously monitors system performance metrics including:

- Operation latency calculated using a moving average
- System resource utilization (CPU, memory, disk)
- User and topic growth statistics

Metrics are logged to `stats.log` every 2 seconds.

## VI. Performance Evaluation

For the sake of performance optimization we adaptively changed the refresh rate of the clients, reducing it to 500ms when the latency exceeded certain values

The server was initialized on an EC2 instance and stress tested to evaluate the performance under heavy load.

For this we utilized the `stress_test.py` script to create 100 users logged into the same topic (worst case scenario), each posting a random message every second and constantly refreshing.

Throughout the stress tested the average operation evidenced a steady increase (considering that it stays around the single digits during low user activity).

Despite the perceived high latency, the overall mesage experience, tested on a different topic so as to avoid the flood

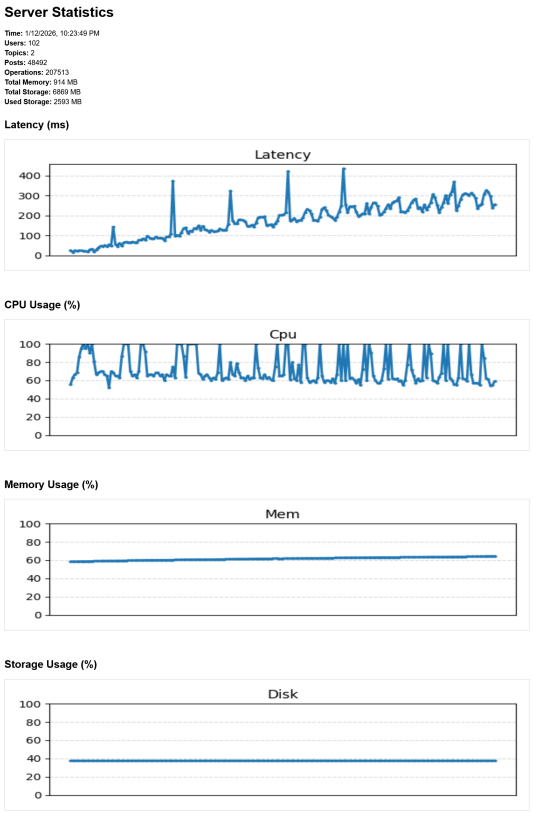



Fig. 2: Server stats page after the stress test, lasting 8 minutes.

of randomized messages, was overall quite acceptable. One thing we did not implement (time constraints) that would make this even smoother would be if the publishing action would preemptively place the user's post on the screen, before the need for a refresh, this would fully eliminate any perceived latency.

When the stress test was performed with 10 clients posting 1 message per second (to simulate a health conversation in a topic) the interface appeared to be highly responsive.

In terms of the CPU usage, it is quite high, possibly due to the quality of the EC2 instance being deployed (t3.micro)

Memory usage showcased a steady, if slow, increase, this is however somewhat meaningless when one observes the very limited ĨGB of RAM allocated to the server.

As we can see despite the large number of posts added during this period the disk usage has barely been affected, showcasing the efficiency of the logging system for storage at this level of scaling.

### A. Scalability and Limitations

Key limitations identified include:

- **Centralized Bottleneck:** All operations pass through a single server process, adding too many users very quickly degrades the server's performance, both through the increase of the limited hardware's load and the increase in

the amount of time that each thread is waiting for the necessary locks.

- **Polling Overhead:** Client refresh operations use periodic polling rather than push notifications, with too many users looking at the same topic this puts far too much pressure on the server.

## VII. Conclusion and Future Work

This paper presented Social4CSE, a comprehensive publish-subscribe system that successfully implements core distributed systems concepts while addressing practical concerns including security, persistence, and monitoring. The system's hybrid cryptographic protocol ensures secure communication, fine-grained locking enables efficient concurrency, and persistent logging provides fault tolerance.

Key achievements include:

- Implementation of a complete pub/sub system with secure communication
- Development of two client interfaces (CLI and web)
- Creation of an administrative monitoring dashboard
- Performance optimizations through smart lock usage and refresh timing logic.

We must acknowledge that our implementations currently has some limitations:

- Cannot have 2 flask client interfaces open on the same browser due to how the session is logged
- Cannot add topics to the server in an easy manner, server has to be shut down, and a new topic log file created, followed by a reboot
- No protocols in place to move data from the EC2 instance for more permanent storage

Future work could be done with any of the following

- **Distributed Architecture:** Extend to multiple server nodes using consistent hashing or consensus protocols
- **Push Notifications:** Replace polling with WebSocket-based push notifications
- **Database Integration:** Replace in-memory storage with distributed databases

Overall, Social4CSE provided a solid foundation for exploring distributed system design and implementation. The project balanced conceptual clarity with practical considerations, making it into an effective learning exercise and a basis for further experimentation and extension.